# وثيقة المراجعة والتلخيص الأكاديمي

## ASE · Chapter 5 — Software Design (Aggarwal & Singh, 3rd ed.)

**المستوى:** Bloom 2 — Understand (فهم/استيعاب). **المعنى:** السؤال ما يطلب حفظًا، يطلب **إعادة صياغة بمفهومك** + تفسير "ليش/كيف" + تمييز بين مفهومين متقاربين. **المصدر:** سلايدات الفصل (99 سلايد) + الكتاب ص.202–251. كل جواب مُتحقَّق من نص الفصل. **قاعدة الإجابة:** إذا جوابك يطابق نص السلايد حرفيًا = أنت تحفظ، مو فاهم. الفهم = تعيد الصياغة + تعطي مثال + تفسّر السبب.

---

## الرسوم المرجعية (راجعها قبل الحل)

### The Two Master Tables

Coupling (between modules) and Cohesion (inside a module) · Aggarwal & Singh, Fig. 7 (slide 19) & Fig. 11 (slide 27)

**COUPLING → aim for the TOP**

6 types · best = Data, worst = Content

BEST

1 · **Data coupling**
BEST — modules exchange only primitive data

2 · **Stamp coupling**
a whole data structure is passed, not just fields

3 · **Control coupling**
a flag is passed to steer the other module's logic

4 · **External coupling**
two modules share an external interface / format

5 · **Common coupling**
both read/write a shared global data area

6 · **Content coupling**
WORST — one module reaches into another's internals

WORST

**COHESION → aim for the TOP**

7 types · best = Functional, worst = Coincidental

BEST

1 · **Functional — BEST (one single task)**

2 · **Sequential (output→input chain)**

3 · **Communicational (same data used)**

4 · **Procedural (same execution order)**

5 · **Temporal (must run at same time)**

6 · **Logical (same logical class)**

7 · **Coincidental — WORST (no relation)**

WORST

**THE GOLDEN RULE (slide 31)**

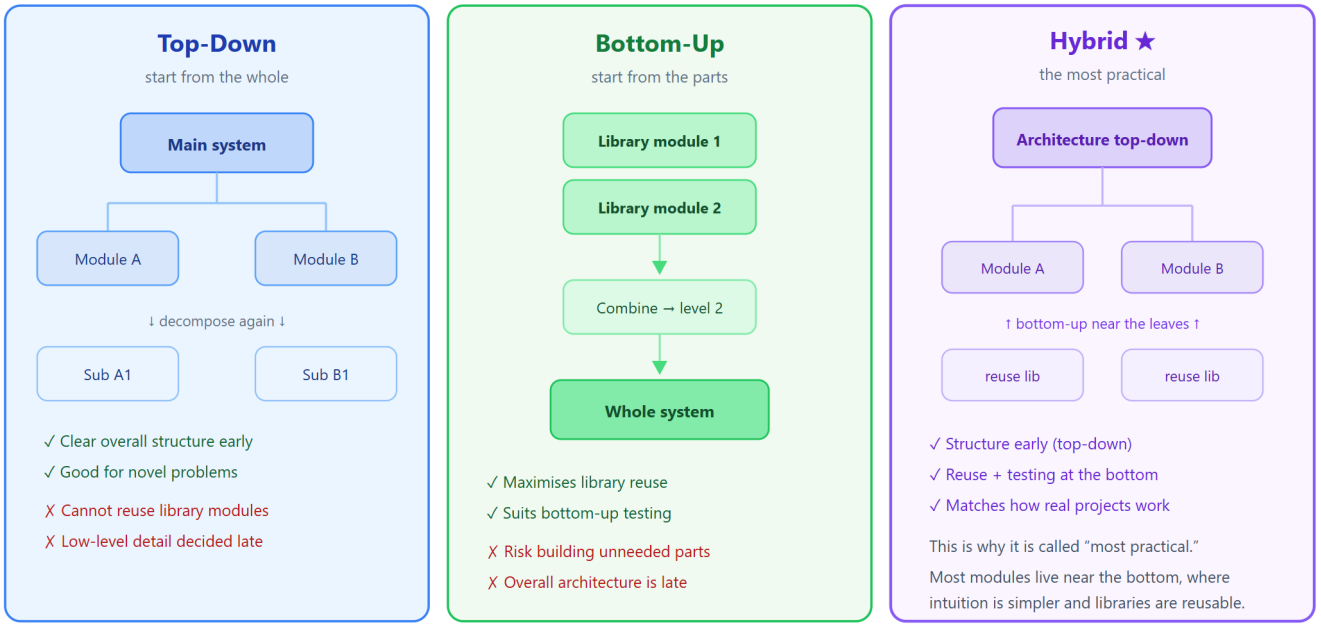
"A software engineer must design the modules with the goal of HIGH cohesion and LOW coupling."

High cohesion = one module · one job · nothing unrelated. Low coupling = few, simple, data-only links between modules.

الجدولان الحاسمان: الاقتران والتلاحم

# Three Strategies of Design

bottom-up · top-down · hybrid (the most practical) · slides 32–35

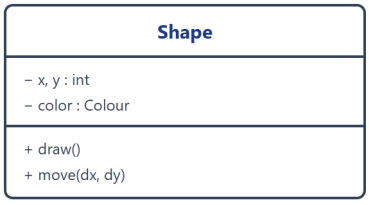



استراتيجيات التصميم الثلاث

# Object-Oriented Design Notation

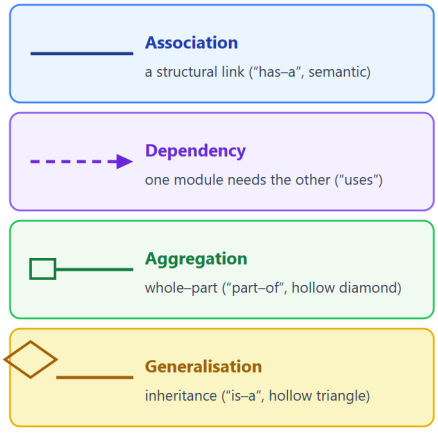
class diagram · the four relationships · object types · slides 63–78

### ANATOMY OF A CLASS

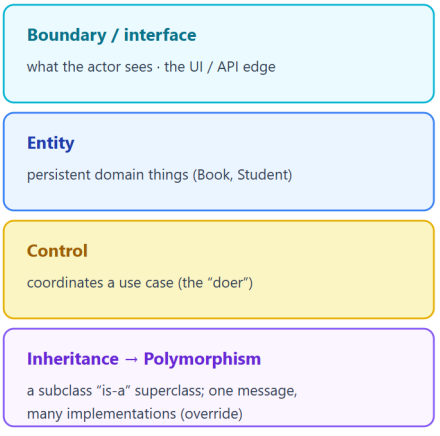



top = name · middle = attributes (state)
bottom = operations (behaviour)

### THE FOUR RELATIONSHIPS (slide 77–78)




### THREE OBJECT TYPES (slide 75)




**Object vs Class vs ADT**

**Object** = an instance with state + behaviour + identity. **Class** = the template from which objects are created.

**ADT** = data + the operations on it, with the implementation hidden. A class extends the ADT by adding inheritance and polymorphism.

Encapsulation hides the state behind the operations — the same principle that turns "content coupling" into an encapsulation violation.

ترميز التصميم الكائني

## كيف تُختبر بالأسئلة (سؤال مهم)

| # | المؤشر | إذا كان جوابك... | الحكم |
|---|---|---|---|
| 1 | تعيد الصياغة بكلماتك | "الارتباط = مستوى اعتماد وحدة على وحدة" | فاهم |
| 2 | تعطي مثال من عندك | "لو دالة تمرّ رقم قومي فقط ⇒ Data" | فاهم |
| 3 | تفسّر السبب | "لأن الاستقلالية تقلل ripple effect" | فاهم |
| 4 | تكرر نص السلايد | "Coupling is the measure of the degree of interdependence..." | حافظ |

---

# القسم A — الفهم التأسيسي (Design · Modularity · Abstraction)

---

**Q1. What does "design is a problem-solving activity" mean, and why is it described as "HOW" rather than "WHAT"?**

**Answer:** Analysis produces the **WHAT** — the requirements, the problem statement, the needs gathered from the user. Design produces the **HOW** — the solution structure that satisfies those requirements. The chapter frames design as "more creative than analysis" (slide 2) because analysis is constrained by what already exists in the problem world, while design invents structure that did not exist. In the two-part design process (slide 5), the customer drives the WHAT (conceptual design) and the system builders drive the HOW (technical design). So "design is HOW" means design is the bridge from the abstract problem to the concrete solution — it decides *which* structure, *which* modules, and *which* interfaces will be used, none of which are given by the requirements themselves.

**التبرير:** يميّز Design عن Analysis (WHAT vs HOW) — مذكور حرفيًا بسلايد 2 و5، وهذا أساس الفصل كله.

---

**Q2. Explain the difference between *conceptual design* and *technical design*. Give one question each type answers.**

**Answer:** The chapter splits design into two parts (Fig. 2, slide 5). **Conceptual design** belongs to the customer — it answers "What will the system look like to users?", "Where will the data come from?", "What choices will be offered?", "How will the reports and screens look?" (slide 6). It is the user's view of the solution. **Technical design** belongs to the system builders — it describes hardware configuration, software needs, communication interfaces, I/O, software architecture, and network architecture (slide 7). Both answer HOW (not WHAT), but at different levels: conceptual design is the *external* behaviour-level HOW, technical design is the *internal* construction-level HOW. Both must together translate requirements into a solution to the customer's problem.

| # | الوجه | Conceptual design | Technical design |
|---|---|---|---|
| 1 | لمن يخدم | العميل (Customer) | البنّاؤون (System builders) |
| 2 | السؤال | Where does data come from? How will screens look? | Hardware config, software architecture, I/O |
| 3 | المستوى | خارجي (ما يراه المستخدم) | داخلي (كيف يُبنى) |

**التبرير:** التمييز بين شقّي التصميم — سلايد 5/6/7. السؤال جدول مقارنة لتفادي تداخل RTL.

---

**Q3. Explain the four objectives of a software design, and why "at the right level" is not the same as "correct".**

**Answer:** The chapter lists four objectives (slide 8): the design must be **Correct & complete** (it satisfies every requirement and leaves nothing out), **Understandable** (a reader can grasp the design without reverse-engineering it), **At the right level** (detail is appropriate — neither too abstract to implement, nor so detailed it becomes code), and **Maintainable** (it can be changed later with contained cost). "At the right level" is not the same as "correct" because correctness is about *content* — does the design satisfy the requirements — while level is about *granularity/reference frame*. A design can be perfectly correct yet still be written at the wrong level of detail: for instance, describing an entire subsystem in one line, or writing actual source statements inside the design document. Both would be "correct" in content but would fail the reader, because neither can be implemented or reviewed properly. Level is a communication property; correctness is a truth property.

**التبرير:** أهداف التصميم الأربعة (سلايد 8) — والسؤال يفصل "صح" عن "بالمستوى الصحيح" لأن الطالب غالبًا يخلط بينهما.

---

**Q4. What is modularity, and why is a module's interface more important than its implementation?**

**Answer:** A modular system consists of well-defined, manageable units with well-defined interfaces among them (slide 11), and the chapter stresses that a module "should be easier to use than to build" and "simpler from outside than from the inside" (slide 12). The interface is more important than the implementation because the interface is the *only* thing the rest of the system depends on. If only the interface is depended upon, the implementation can be changed freely without rippling outward — which is exactly what encapsulation/information hiding achieves (slide 69). The chapter's own coupling scale confirms this: the best coupling (data) depends only on the parameters crossing the interface, while the worst (content) reaches inside an implementation. So a module's value is measured by how cleanly its interface stands alone.

**التبرير:** تعريف Modularity + خصائص الوحدة (سلايد 10–13)، والربط بفكرة "الواجهة = نقطة الاعتماد الوحيدة" اللي تفسر جدول الـcoupling.

---

**Q5. Explain the modularity vs software cost relationship (the U-curve) and what it warns you against.**

**Answer:** The modularity-versus-cost figure (Fig. 4, slide 14) plots "Cost of Effort" against "Number of Modules" and produces a **U-shaped curve**: cost is high when there are too few modules (each module is huge and hard to understand and change), falls to a minimum at an intermediate number of modules (the "Region of Minimum Cost"), then rises again as the number of modules grows further because the **cost-to-integrate** modules begins to dominate — every extra interface adds communication, testing and coordination work. The chart's warning is that "more modules" is not automatically better: modularity has an optimum, and both extremes — under-modularising and over-modularising — are expensive. The designer's job is to sit near the bottom of the U, not at either arm.

**التبرير:** Fig. 4 (سلايد 14) — منحنى U ومعنى Region of Minimum Cost. يمنع الخطأ الشائع "كل ما زادت الوحدات = أفضل".

---

**Q6. What does "abstraction" mean in object-oriented design, and why does the chapter call it "the elimination of the irrelevant and the amplification of the essentials"?**

**Answer:** The chapter defines abstraction as "the elimination of the irrelevant and the amplification of the essentials" (slide 61) — complexity is managed by deliberately discarding the details that do not matter for the current purpose, and by emphasising the few features that do. The phrase captures both halves of the act: it is not merely deletion (elimination) but also *highlighting* (amplification). In OOD this is applied at several scales — the class abstracts the common structure and behaviour of many objects; the interface abstracts an operation's signature while leaving its implementation to subclasses (polymorphism, slide 69); and inheritance pulls the important parts of several classes up into a common superclass (Fig. 23, slide 68). In every case the point is: the reader must be able to think about the design without carrying irrelevant detail in working memory.

**التبرير:** تعريف Abstraction (سلايد 61) — والسؤال يشرح سبب الصياغة المزدوجة "amplification + elimination" وربطها بأشكال 21–23.

---

**Q7. Explain the difference between *data abstraction* and *procedural abstraction*.**

**Answer:** Both are abstraction mechanisms but they hide different things. **Procedural (functional) abstraction** hides the *algorithm* behind a named operation: the caller writes sort(records) and need not know which sort is used; the notation for this in the chapter is the structure chart's black box — "a black box means that functionality is known to the user without the knowledge of internal design" (slide 41) — and pseudocode at the statement level (slide 46). **Data abstraction** hides the *representation* of a datum behind a named type plus the set of operations allowed on it: the caller knows there is a "Student record" and what can be done with it, but not how the fields are laid out or stored. An **abstract data type** is precisely a data abstraction whose permitted operations are defined but whose representation is hidden — and the chapter's Exercise 5.17 asks the relationship between ADTs and classes, which is that a class is an ADT plus inheritance and the object model.

| # | الوجه | Procedural abstraction | Data abstraction |
|---|---|---|---|
| 1 | ما يُخفى | الخوارزمية/الطريقة | التمثيل الداخلي للبيانات |
| 2 | الشكل في الفصل | Structure chart black box (سلايد 41) | Abstract Data Type / class |
| 3 | المثال | sort(records) | Student بخصائصه وعملياته فقط |

**التبرير:** فرق أساسي بالسلايد 41/46/61 — والسؤال جدول مقارنة حتى لا يتشوش الترتيب RTL.

---

**Q8. What is *information hiding* (encapsulation), and why is it usually listed as the same idea as "the module's interface is its contract"?**

**Answer:** The chapter defines encapsulation as "the separation of the external aspects of an object from the internal implementation details of the object" (slide 69) and notes it is commonly referred to as **information hiding**. The reason it is the same idea as "the interface is its contract" is that once the external aspect is separated out, that external aspect *is* the only thing the rest of the system can legally depend on — in other words it becomes a contract. Internal implementation details are no longer visible, so nobody can couple to them, and content coupling becomes impossible by construction. This is why encapsulation is not just a nicety: it is the mechanism that makes the best coupling (data, through the interface) the *only* available coupling. Protecting the module's internals is the direct cause of reducing inter-module dependency.

**التبرير:** تعريف Encapsulation (سلايد 69) — والسؤال يربطها بجدول الـcoupling: تمنع Content coupling بنيويًا.

---

# القسم B — الفهم العميق: Coupling

**Q9. What does coupling measure, and why is "no dependencies at all" not achievable in practice?**

**Answer:** Coupling is the measure of the degree of interdependence between modules (slide 15). The chapter pictures three situations: uncoupled (no dependencies), loosely coupled (some dependencies) and highly coupled (many dependencies) (Fig. 5, slides 15–16). "No dependencies at all" is not achievable for any system that does real work, because useful systems require modules to cooperate — a module that computes nothing and talks to nobody is useless. Moreover, the chapter's own Exercise 5.24 explicitly asks whether a system can ever be completely decoupled and whether coupling can be reduced until there is *no* coupling between modules. The realistic target is therefore *minimum* coupling, not zero coupling: the designer reduces dependencies down toward the data-coupling end without eliminating the necessary cooperation between modules.

**التبرير:** تعريف Coupling (سلايد 15–16) + تمرين 5.24 — "لا يوجد صفر coupling"، الهدف تقليله لا إلغاؤه.

---

**Q10. Explain the six types of coupling in order from best to worst, with one sentence each.**

**Answer (best → worst):** The chapter fixes the order in Fig. 7 (slide 19).

| # | النوع | الترتيب | المعنى بمجرد جملة |
|---|---|---|---|
| 1 | Data coupling | Best (الأفضل) | وحدتان تتواصلان فقط بتمرير بيانات، وما عدا ذلك مستقلتان (سلايد 20). |
| 2 | Stamp coupling | 2 | تُمرَّر بنية بيانات كاملة بين الوحدتين، أكثر مما تحتاج الوحدة المستقبِلة (سلايد 20). |
| 3 | Control coupling | 3 | التواصل بتمرير معلومة تحكّم (flag) تُضبط في وحدة ويُتفاعل معها التابع (سلايد 21). |
| 4 | External coupling | 4 | ارتباط بترتيب/صيغة خارجية مفروضة على النظام (سلايد 19). |
| 5 | Common coupling | 5 | وحدتان تتشاركان منطقة بيانات عامة (Global) — تغييرها يتطلب تتبّع كل من يصل إليها (سلايد 21). |
| 6 | Content coupling | Worst (الأسوأ) | وحدة تغيّر بيانات وحدة أخرى أو يُمرَّر التحكّم إلى وسط وحدة أخرى (سلايد 23). |

**التبرير:** جدول Fig. 7 (سلايد 19–21) — الترتيب مثبّت بالسهم "Best" فوق و"Worst" تحت. الجدول يمنع تلخبط RTL.

---

**Q11. Why is Data coupling the best and Content coupling the worst? Give the single principle that orders the whole scale.**

**Answer:** Data coupling is the best because the two modules communicate by passing data *only* — and apart from that data channel they are independent (slide 20). Nothing hidden is shared, the dependency is one-directional, and the caller can see exactly what it is sending. Content coupling is the worst because the module reaches into another module's internals — "module A changes data of module B, or control is passed from one module to the middle of another" (slide 23), where module B branches into D even though D is supposed to be under C's control. The single ordering principle is: **how much hidden, internal knowledge of the other module is required, and how bidirectional the dependency is**. Passing the minimum data requires the least knowledge and is most one-directional (best); reaching into internals requires the most knowledge and creates an invisible bidirectional link (worst). Every intermediate type sits between these two extremes on that same axis.

**التبرير:** سلايد 20 + 23، ومبدأ الترتيب ورد أيضًا في Tier2 Q5.2 — أقل معرفة داخلية + أحادي الاتجاه = الأفضل.

---

**Q12. What is the difference between *stamp coupling* and *data coupling*, and why does passing a whole record hurt?**

**Answer:** Data coupling passes *only the data actually needed* — typically simple values, one datum per parameter. Stamp coupling passes a *complete data structure* — an entire record or object — from one module to another (slide 20). Passing the whole record hurts because the receiving module now has access to far more than it requires: it can read, and in careless designs even depend on, fields it has no business knowing about. That extra exposure means (a) the receiving module's behaviour can silently begin to depend on unrelated fields, and (b) if the record's layout changes, the receiving module may need to change even though the datum it actually used did not change. The chapter's Fig. 6 example (slide 18) favours the "loose coupling" design that passes only student ID rather than the whole student record. Stamp coupling is still better than control, external, common or content coupling, but it is worse than passing exactly what is needed.

**التبرير:** سلايد 20 + مثال Fig. 6 (سلايد 18) — تمرير الـrecord كامل يُعرّض بيانات لا تحتاجها الوحدة.

---

**Q13. Explain *control coupling* and why a flag makes two modules depend on each other's logic.**

**Answer:** Control coupling occurs when modules communicate by passing *control information* — "usually accomplished by means of flags that are set by one module and reacted upon by the dependent module" (slide 21). A flag makes the two modules depend on each other's logic because the called module must contain a *branch* that knows what to do for each flag value, and the calling module must *know* which flag values are meaningful. Neither module can be understood alone: the caller's correct behaviour depends on the callee's internal branching, and the callee's internal branching is only exercised by the caller's flag choices. The chapter's guidance is explicit — "Pass data, not the control information" (slide 17). Replacing a flag with two separate, purpose-named modules removes the branch entirely and drops the coupling back toward data.

**التبرير:** تعريف Control coupling (slide 21) + القاعدة الصريحة "Pass data, not control information" (slide 17).

---

**Q14. Explain *common coupling* using the "global data area" example, and why the chapter says a change forces "tracing back to all modules".**

**Answer:** With common coupling, modules A and B have *shared data* — "global data areas are commonly found in programming languages", and "making a change to the common data means tracing back to all the modules which access that data to evaluate the effect of changes" (slide 21, with an example in Fig. 8, slide 22). A change forces tracing back to all accessing modules because the global area has no single owner and no visible interface: nobody can tell, simply by reading a module, which other modules depend on each value they touch. Any modification therefore requires an exhaustive search of every module that reads or writes the shared area. This is why common coupling ranks fifth — worse than external coupling but better than content coupling: the dependency is real and bidirectional, but at least it is a *named shared region* rather than another module's private internals.

**التبرير:** تعريف Common coupling (سلايد 21) + Fig. 8 (سلايد 22) — المنطقة العامة بلا مالك واحد = تعقّب كل الوصولات عند أي تغيير.

---

**Q15. Explain *content coupling* with the chapter's own example, and why it is called "invisible change".**

**Answer:** Content coupling occurs when module A changes data of module B, or when control is passed from one module to the middle of another; in the chapter's Fig. 9 (slide 23), "module B branches into D, even though D is supposed to be under the control of C." It is called invisible change because the dependency does not appear anywhere in a parameter list or a shared declaration — module B simply reaches into D's or C's internals out of band, so no interface, compiler or reviewer can see the link. Consequently, changing C or D can silently break B with no compile-time signal, and the ripple effect becomes uncontainable. This is the worst coupling (slide 19) precisely because it destroys encapsulation: the separations of interfaces that all the other coupling types at least respect are violated outright.

**التبرير:** تعريف Content coupling (سلايد 23) + Fig. 9 — لا تظهر بالواجهة، لذلك التغيير يكون صامتًا (ripple غير محتوى).

---

# القسم C — الفهم العميق: Cohesion

**Q16. What does cohesion measure, and how is it different in *direction* from coupling?**

**Answer:** Cohesion is "a measure of the degree to which the elements of a module are functionally related" (slide 25) — how strongly the parts *inside one module* belong together. Coupling measures the interdependence *between* modules (slide 15). The directional difference is therefore intra-module versus inter-module: cohesion looks *inward* at one module's internal strength of relation, coupling looks *outward* between two modules. They are two views of the same partition of the system (Fig. 12, slide 31): splitting a module badly — low cohesion — produces pieces that still need to cooperate, which forces extra links and raises coupling; conversely, a badly placed module boundary forces unrelated elements into one unit and lowers cohesion in both. The chapter's stated goal is therefore always "high cohesion and low coupling" together (slide 31).

| # | الوجه | Cohesion | Coupling |
|---|---|---|---|
| 1 | الاتجاه | داخل الوحدة (intra-module) | بين الوحدتين (inter-module) |
| 2 | يقيس | قوة ترابط عناصر الوحدة | درجة اعتماد وحدة على وحدة |
| 3 | المصدر | سلايد 25 | سلايد 15 |
| 4 | الهدف | عالية | منخفضة |

**التبرير:** تعريف Cohesion (سلايد 25) مقابل Coupling (سلايد 15) + Fig. 12 (سلايد 31) — سؤال جدول لتجنّب تداخل RTL.

---

**Q17. Explain the seven types of cohesion from best to worst, with one sentence each.**

**Answer (best → worst):** The chapter fixes the order in Fig. 11 (slide 27).

| # | النوع | الترتيب | المعنى بمجرد جملة |
|---|---|---|---|
| 1 | Functional cohesion | Best (الأفضل) | A و B جزءان من مهمة وظيفية واحدة (سلايد 28). |
| 2 | Sequential cohesion | 2 | مخرجات خطوة تُصبح مُدخلات الخطوة التالية (سلايد 28). |
| 3 | Communicational cohesion | 3 | عناصر الوحدة تعمل على *نفس البيانات* (مُدرّجة في جدول سلايد 27). |
| 4 | Procedural cohesion | 4 | مهام مختلفة لكن مجموعة بسبب *ترتيب* تنفيذ محدد (سلايد 29). |
| 5 | Temporal cohesion | 5 | مهام مرتبطة بأنها تُنفَّذ في *نفس الفترة الزمنية* (سلايد 29). |

| # | النوع | الترتيب | المعنى بمجرد جملة |
|---|---|---|---|
| 6 | Logical cohesion | 6 | عناصر تبدو مرتبطة لأنها من *نفس الصنف المنطقي* من الدوال (سلايد 30). |
| 7 | Coincidental cohesion | Worst (الأسوأ) | عناصر بينها علاقة ضئيلة أو معدومة (سلايد 30). |

**التبرير:** جدول Fig. 11 (slide 27) — 7 مستويات، "Best (high)" فوق و"Worst (low)" تحت. ملاحظة: Communicational موجود في جدول Fig. 11 لكن نصّ الشرح التفصيلي يغطي الستة الباقية (slide 26).

---

**Q18. What is the difference between *procedural cohesion* and *sequential cohesion*?**

**Answer:** Sequential cohesion exists when one element's *output is the input of the next* — the chapter's words: "Module A outputs some data which forms the input to B. This is the reason for them to be contained in the same procedure" (slide 28). Procedural cohesion exists when the instructions "although accomplish different tasks, yet have been combined because there is a specific order in which the tasks are to be completed" (slide 29). The difference is *data flow versus control flow*: sequential cohesion is welded together by actual data passing from step to step, while procedural cohesion is welded together only by the order of execution — the steps could run in that order but no data crosses between them. That is why sequential sits higher (position 2) than procedural (position 4): a real data dependency is a stronger reason for two elements to live in the same module than a merely conventional order.

**التبرير:** سلايد 28 مقابل سلايد 29 — الفرق = تدفّق بيانات (Sequential) أم ترتيب تنفيذ فقط (Procedural).

---

**Q19. Why does *logical cohesion* usually force control coupling on the caller?**

**Answer:** Logical cohesion occurs in modules containing instructions "that appear to be related because they fall into the same logical class of functions" (slide 30) — for example, a module containing every input routine. Because the module bundles several *different* actual tasks under one name, the caller must tell the module at runtime *which* task it wants, and in practice this is done by passing a control flag or selector value into the module. That selector *is* control coupling (slide 21). So logical cohesion at the level of the callee directly produces control coupling at the level of the interface — the two defects come as a package. This is also exactly what the chapter's Exercise 5.8 asks: "If a module has logical cohesion, what kind of coupling is this module likely to have with others?" The answer is control coupling, because the module cannot decide its own task.

**التبرير:** سلايد 30 (Logical) + سلايد 21 (Control) + تمرين 5.8 — العلاقة المباشرة بينهما سؤال امتحاني متكرر.

---

**Q20. Explain the relationship between cohesion and coupling as shown in Fig. 12, and why they are not independent.**

**Answer:** Fig. 12 (slide 31) presents the two properties as two views of the same modular partition, with the textual rule "a software engineer must design the modules with goal of **high cohesion and low coupling**". They are not independent because they are produced by the *same* decision: where to draw the module boundaries. Draw a boundary through the middle of a functional task and you get two weakly cohesive modules that still need to pass data back and forth, so coupling rises. Draw a boundary that wraps a whole functional task and its exclusive data, and you get one highly cohesive module with a small data-only interface, so coupling falls. Hence the chapter's warning that if the software is not properly modularised, "a host of seemingly trivial enhancements or changes will result into the death of the project" (slide 31). Improving cohesion correctly is the same act as reducing coupling.

**التبرير:** Fig. 12 (سلايد 31) — الاثنان نتاج القرار نفسه: مكان حدّ الوحدة. يفسّر ليش ممنوع تحسين أحدهما بمعزل عن الآخر.

---

# القسم D — الفهم العميق: SDD · Notations · Strategies

**Q21. Explain the difference between top-down and bottom-up design, and why the chapter concludes that hybrid is the practical answer.**

**Answer:** Top-down design "starts by identifying the major modules of the system, decomposing them into their lower level modules and iterating until the desired level of detail is achieved" (slide 34) — stepwise refinement from an abstract design to a concrete one. Bottom-up design instead builds and collects lower-level modules first, gathering them "in the form of a library" (slide 33), and composes upward. The chapter argues hybrid is the practical answer (slide 35) for three reasons: it permits **common sub-modules**; near the **bottom** of the hierarchy intuition is simpler and bottom-up testing matters more because there are more modules at low levels; and it allows **reuse of pre-written library modules**. In practice the top-down view fixes the major structure early while bottom-up assembly handles the dense lower levels.

| # | الوجه | Top-down | Bottom-up |
|---|---|---|---|
| 1 | نقطة البداية | الوحدات الرئيسية (abstract) | الوحدات السفلى (library) |
| 2 | الأسلوب | Stepwise refinement | تجميع من مكتبة موجودة |
| 3 | القوة | يثبّت البنية الكبرى مبكرًا | يدعم إعادة الاستخدام والاختبار السفلي |
| 4 | المصدر | سلايد 34 | سلايد 33 |

**التبرير:** سلايد 33/34/35 — Hybrid (توليفة الاثنين) هو الأنسب عمليًا، وهذا مثبّت في الحقائق المتحققة.

---

**Q22. Why is a *structure chart* fundamentally different from a *flowchart*?**

**Answer:** A flowchart describes the **flow of control and sequence of processing** — the order in which operations execute, with decisions and loops. A structure chart "partitions a system into block boxes", where "a black box means that functionality is known to the user without the knowledge of internal design" (slide 41, Fig. 16). The essential difference is therefore *what* each notation is about: the structure chart shows the **hierarchical decomposition into modules and the interfaces (parameters) between them** — a static, structural, data-flow view of the design — while the flowchart shows **temporal execution order** inside or across processing steps — a dynamic, control-flow view. This is why the chapter's Exercise 5.13 asks precisely for the difference between a flow chart and a structure chart: they are not interchangeable notations, they answer different questions about the same system.

| # | الوجه | Structure chart | Flowchart |
|---|---|---|---|
| 1 | يجيب عن | كيف تتقسّم الوحدات وما يُمرَّر بينها | بأي ترتيب تنفّذ العمليات |
| 2 | النوع | Static / structural | Dynamic / control-flow |
| 3 | العنصر | Black box module + parameters | Process + decision + loop |
| 4 | المصدر | سلايد 41 | تمرين 5.13 |

**التبرير:** سلايد 41 + تمرين 5.13 — فرق "بنية ثابتة" مقابل "تسلسل تنفيذ".

---

**Q23. Explain what the SDD is and why the chapter calls it "the blueprint for the implementation activity".**

**Answer:** An SDD (Software Design Description) is "a representation of a software system that is used as a medium for communicating software design information" (slide 50) and is created "to facilitate analysis, planning, implementation and decision making" (slide 51). The chapter states that "the SDD shows how the software system will be structured to satisfy the requirements identified in the SRS… it is basically the translation of requirements into a description of the software structure, software components, interfaces, and data necessary for the implementation phase. Hence, SDD becomes the blue print for the implementation activity" (slide 52). Calling it a blueprint means it is the authoritative reference that implementers build *from* and that correctness is judged *against* — it is not a summary produced after the fact but the source specification for coding.

**التبرير:** تعريف SDD (سلايد 50–52) — الاستعارة "blueprint" مذكورة حرفيًا بسلايد 52.

---

**Q24. Explain the four design views of the SDD, and why more than one view is needed at all.**

**Answer:** Table 2 (slide 57) organises the SDD into four views. **Decomposition description** — partitions the system into design entities (e.g. hierarchical decomposition diagram, natural language). **Dependency description** — describes relationships among entities (structure chart, data flow diagrams, transaction diagrams). **Interface description** — everything a designer, developer or tester needs in order to *use* each entity (interface files, parameter tables). **Detail description** — the internal design details of an entity (flow charts, PDL). More than one view is needed because, as the chapter says, "each design description writer may have a different view of what are considered the essential aspects of a software design" (slide 54), and different project activities need different subsets: a tester needs interfaces, an implementer needs details, an architect needs the decomposition. A single flat document would force every reader to wade through irrelevant information.

| # | Design View | Scope | مثال التمثيل |
|---|---|---|---|
| 1 | Decomposition description | تقسيم النظام لوحدات | Hierarchical decomposition diagram |
| 2 | Dependency description | علاقات الوحدات ببعضها | Structure chart, DFD |
| 3 | Interface description | كل ما يحتاجه المستخدم للوحدة | Interface files, parameter tables |
| 4 | Detail description | التفاصيل الداخلية للوحدة | Flow charts, PDL |

**التبرير:** Table 2 (سلايد 57) — أربع views، وسبب التعدد مذكور بسلايد 54 (كل قارئ له وجهته).

---

**Q25. Explain what the ten design entity attributes are, and why an entity needs all of them rather than just a name.**

**Answer:** The chapter lists ten attributes of a design entity (slides 53): Identification, Type, Purpose, Function, Subordinates, Dependencies, Interface, Resources, Processing, and Data. A name alone is insufficient because a design entity is "an element of a design that is structurally and functionally distinct from other elements and that is separately named and referenced" (slide 51) — being named is only one of its properties, not its whole identity. Each attribute answers a different question a downstream reader will ask: *Identification* — which entity is this; *Type* — what kind of entity; *Purpose* — why it exists; *Function* — what it does; *Subordinates* — what it decomposes into; *Dependencies* — what it relies on; *Interface* — how to use it; *Resources* — what it consumes;

*Processing* — how it works internally; *Data* — what data it handles. Together they form the "statement of fact about the entity" (slide 51), which is what makes the SDD usable by implementers, testers and maintainers.

**التبرير:** سلايد 53 + تعريف Entity attributes (سلايد 51) — عشرة، وكل واحد يجيب سؤالًا مختلفًا لقارئ مختلف.

---

# القسم E — الفهم العميق: Object-Oriented Design

**Q26. What is an *object* in OOD, and how is it different from a *class*?**

**Answer:** An object is "an entity able to save a state (information) and which offers a number of operations (behavior) to either examine or affect this state. An object is characterized by number of operations and a state which remembers the effect of these operations" (slide 60). A class is "a set of objects that share a common structure and a common behavior" (slide 62) — "classes are useful because they act as a blueprint for objects" (slide 62). The difference is instance versus description: the class is the template that specifies which attributes and operations exist, and each object is one *instance* of that template carrying its own attribute values — the chapter's example being Indica, Santro, Maruti and Indigo as instances of the class "car" (Fig. 20, slide 63). All objects of a class share the same operations, but each holds its own state.

| # | الوجه | Object | Class |
|---|---|---|---|
| 1 | الطبيعة | نسخة (instance) فعلية | قالب/blueprint |
| 2 | الحالة | له قيم خصائص خاصة به | يحدد الخصائص والعملية المشتركة |
| 3 | المثال في الفصل | Indica, Santro | Class "car" |
| 4 | المصدر | سلايد 60 | سلايد 62 |

**التبرير:** سلايد 60 + 62 + Fig. 20 (سلايد 63) — instance مقابل blueprint.

---

**Q27. What is an *abstract data type*, and what is its relationship to a class?**

**Answer:** An abstract data type (ADT) is a data abstraction whose *permitted operations* are defined while its *internal representation* is hidden — the user of the type knows what can be done with it, not how it is stored. A class is an ADT enriched with the object model: the chapter defines a class as a set of objects sharing a common structure and behaviour (slide 62), with attributes as "a data value held by the objects in a class" (slide 65) and operations as "a function or transformation that may be applied to or by objects in a class" (slide 65). The relationship is therefore that a class *is* an ADT plus the additional mechanisms of inheritance, polymorphism and the notion of an instance — which is exactly what the chapter's Exercise 5.17 asks: "What is the relationship between abstract data types and classes?" Classes implement the ADT idea concretely.

**التبرير:** سلايد 62/65 + تمرين 5.17 — ADT = Class + inheritance/polymorphism/instances.

---

**Q28. Explain *inheritance* using the chapter's Shape example, and state precisely what is inherited.**

**Answer:** Inheritance is described as the mechanism by which "low level classes (known as subclasses or derived classes) inherit state and behavior from this high level class (known as a super class or base class)" (slide 68). In the chapter's example, the Square and Triangle classes (Figs. 21–22) differ in detail but, "at a high level of abstraction, we might want to think of a picture as made up of shapes… we do not care that one shape is a

square and the other is a triangle as long as both can draw themselves" (slide 67). The important parts are therefore pulled out into a new class called **Shape** (Fig. 23, slide 68), and the subclasses inherit from it. Precisely, what is inherited is **state (attributes) and behaviour (operations)** — not implementation choices made inside the subclasses. This lets the design express sub-types generically, which the chapter links to hierarchy as a complexity-reducing mechanism (slide 69).

**التبرير:** سلايد 67/68 + Figs. 21–23 — الموروث = behaviour + state من الـsuperclass.

---

**Q29. Explain the difference between *inheritance* and *polymorphism*, and why one can exist without the other.**

**Answer:** Inheritance is a *structural* relationship — subclasses derive state and behaviour from a superclass (slide 68). Polymorphism is a *behavioural* mechanism — "when we abstract just the interface of an operation and leave the implementation to subclasses it is called a polymorphic operation and process is called polymorphism" (slide 69). The chapter's Exercise 5.18 asks directly: "Can we have inheritance without polymorphism? Explain." The answer is yes: a subclass can inherit state and behaviour from a superclass without any operation being abstracted for overriding at a subclass — the inherited behaviour simply runs. The reverse is also conceptually separable: polymorphism requires a common type and an overridden operation, which typically comes from an inheritance hierarchy, but the *essence* of polymorphism (same message, different implementations) is the interface/implementation split, not the derivation itself.

| # | الوجه | Inheritance | Polymorphism |
|---|---|---|---|
| 1 | الطبيعة | علاقة بنيوية (subclass ← superclass) | آلية سلوكية (نفس الواجهة، تنفيذات مختلفة) |
| 2 | ما يحدث | يرث behaviour + state | يُجرَّد interface وتُترك implementation للـsubclass |
| 3 | السؤال الرسمي | سلايد 68 | سلايد 69 + تمرين 5.18 |

**التبرير:** سلايد 68/69 + تمرين 5.18 — يمكن وجود وراثة بلا polymorphism لأن الوراثة بنيوية والتعددية سلوكية.

---

**Q30. Explain *aggregation*, *association* and *dependency* as class relationships, and how they differ in strength.**

**Answer:** The chapter lists four relationship types in class diagrams (slides 77–78). An **association** is a "semantic connection between classes" — when it connects two classes, each can send messages to the other in a sequence or collaboration diagram, and it may be bi-directional or unidirectional. A **dependency** is "always unidirectional" and "shows that one class depends on the definitions in another class." An **aggregation** is a "stronger form of association" and is "a relationship between a whole and its parts." The strength ordering implied is dependency < association < aggregation, with generalisation (inheritance) as the fourth type. The key discriminator is what the link means: dependency is a one-way "needs the definition" link with no structural containment; association is a two-way or one-way semantic link between peers; aggregation is ownership/containment — the whole *has* the part.

| # | العلاقة | المعنى | الاتجاه |
|---|---|---|---|
| 1 | Dependency | وحدة تعتمد على تعريفات وحدة أخرى | أحادي الاتجاه دائمًا |
| 2 | Association | رابط دلالي بين صنفين، تبادل رسائل | ثنائي أو أحادي |

| # | العلاقة | المعنى | الاتجاه |
|---|---|---|---|
| 3 | Aggregation | علاقة "كل بجزئه" (whole-part) | أقوى من Association |
| 4 | Generalization | وراثة بين صنفين | من الأخص إلى الأعم |

**التبرير:** سلايد 77/78 (أربعة أنواع) — القوة: Dependency أضعف، ثم Association، ثم Aggregation (whole-part) أقوى).

---

**Q31. Explain what a *use case model* captures, and why it is the first step of the OOD process.**

**Answer:** The first step in the analysis and design of an object-oriented system (Fig. 25, slide 72) is to "create use case model", and the chapter explains it: "first step is to identify the actors interacting with the system. We should then write the use case and draw the use case diagram" (slide 73). A use case model therefore captures *who* interacts with the system (the actors) and *what goals* they pursue (the use cases), plus the system boundary that separates them. It comes first because everything downstream depends on the boundary it fixes: the interaction diagram needs the objects of each use case identified, the class diagram needs the entities those use cases manipulate, and the state chart needs the objects whose lifecycles matter. In other words, establishing actors and system scope first prevents the model from growing unbounded, which is the model-level equivalent of avoiding coincidental cohesion.

**التبرير:** سلايد 72/73 — Step 1 من 6 خطوات OOD، ويحدد الـactors وحدود النظام قبل أي شيء آخر.

---

**Q32. Explain the difference between a *use case diagram* and a *sequence diagram*.**

**Answer:** A use case diagram, produced in step 1 of the OOD process (slide 73), shows the **static scope of interaction** — the actors outside the system and the goals/use cases the system provides for them; it says nothing about ordering or internal messages. A sequence diagram is one of the **interaction diagrams** produced in step 3 (slide 75), and "an interaction diagram shows an interaction, consisting of a set of objects and their relationship, including the messages that may be dispatched among them. Interaction diagrams address the dynamic view of a system" (slide 75). So the difference is scope-and-structure versus time-and-messages: the use case diagram answers "what can be done and by whom", while the sequence diagram answers "in what order do these specific objects exchange which messages to accomplish one use case". The chapter explicitly says sequence and collaboration diagrams are drawn *for every use case*.

| # | الوجه | Use case diagram | Sequence diagram |
|---|---|---|---|
| 1 | المرحلة | Step 1 (سلايد 73) | Step 3 (سلايد 75) |
| 2 | يعرض | Actors + use cases + حدود النظام | Objects + messages + ترتيب زمني |
| 3 | النوع | Static scope | Dynamic view (زمني) |
| 4 | السؤال | ماذا يمكن أن يُفعل ولمن؟ | بأي ترتيب تتبادل الكائنات الرسائل؟ |

**التبرير:** سلايد 73 مقابل 75 — الأول نطاق ثابت، الثاني عرض ديناميكي بأزمنة ورسائل.

---

**Q33. Why does the chapter say OOD is "orthogonal" to function-oriented design, and what does each begin with?**

**Answer:** "Object oriented design is the result of focusing attention not on the function performed by the program, but instead on the data that are to be manipulated by the program. Thus, it is **orthogonal** to function oriented design" (slide 58). Orthogonal means the two decompositions run along *different axes* of the same system: **function-oriented design** decomposes by *what the system does* — "the design is decomposed into a set of interacting units where each unit has a clearly defined function… system is designed from a functional viewpoint" (slide 36). **OOD** "begins with an examination of the real world 'things' that are part of the problem to be solved. These things (which we will call objects) are characterized individually in terms of their attributes and behavior" (slide 58). So function-oriented design begins with the processes; OOD begins with the real-world entities and their states. Neither is a refinement of the other — they are perpendicular views.

**التبرير:** سلايد 58 + 36 — Orthogonal = محورين متعامدين: الدوال (FO) مقابل الكيانات (OO). تمرين 5.22 يطلب هذه المقارنة.

---

**Q34. Explain the reasons the chapter gives for improvement using object-oriented design.**

**Answer:** The chapter's Exercise 5.19 asks "Discuss the reasons for improvement using object-oriented design," and the answers are grounded in the definitions of slides 58–69. First, OOD **begins from the real-world things of the problem** (slide 58), which are stable relative to the operations performed on them — so requirements changes that alter behaviour attach to existing entities rather than forcing a redesign of the decomposition. Second, **encapsulation** separates the external aspects of an object from its internal implementation (slide 69), so a change inside one object cannot silently break others — this directly bounds the ripple effect. Third, **inheritance and hierarchy** let common state and behaviour be expressed once and sub-types be treated generically (slides 68–69), which reduces duplication. Fourth, **polymorphism** abstracts the interface while leaving implementation to subclasses (slide 69), so new behaviour is added by adding a subclass rather than by editing existing code.

**التبرير:** تمرين 5.19 مع سلايد 58/68/69 — أربعة أسباب: البداية من الكيانات، ثم Encapsulation، ثم Inheritance/Hierarchy، ثم Polymorphism.

---

**Q35. Explain why the OOD process has six steps and why each earlier step constrains the next.**

**Answer:** The chapter presents the analysis and design of an object-oriented system as six steps (Fig. 25, slides 72 and 75): (1) Create use case model, (2) Draw activity diagram (if required), (3) Draw the interaction diagrams, (4) Draw the class diagram, (5) Design of state chart diagrams, (6) Draw component and deployment diagram. Each earlier step constrains the next because each fixes a different layer of the model. Step 1 fixes the *actors and scope*, so step 3 knows which objects need to be identified for each use case ("firstly, we should identify the objects with respect to every use case", slide 75). Step 3 fixes the *message traffic*, which reveals what state the objects must hold, feeding step 4's class diagram. Step 4 fixes the *entities and their operations*, whose lifecycles step 5 then elaborates in state charts. Step 6 finally maps classes onto physical components (slide 80). Skipping forward is impossible without inventing the missing layer.

**التبرير:** Fig. 25 (سلايد 72/75) — ست خطوات مثبّتة في الحقائق المتحققة، وكل خطوة تُغذّي التالية بالمعلومات.

---

# القسم F — اختبار الجملة الواحدة (One-Sentence Test)

---

## F1 — اختبار الـCoupling (جملة واحدة)

**السؤال:** كيف تعرف في جملة واحدة أن الارتباط **سيئ**؟

**الجواب (جملة واحدة):** الارتباط سيئ إذا كانت الوحدتان **تتقاسمان شيئًا مخفيًا** (بيانات عامة، ترتيبًا خارجيًا، داخل وحدة أخرى) — يعني الأسوأ كل ما كان "sharing" أكثر من "passing"، والأفضل كل ما كان "passing" للبيانات الضرورية فقط أحادي الاتجاه.

**التشخيص السريع:** اسأل نفسك — *هل الوحدة تتشارك موردًا أم تمرّر بيانات؟* تشارك ⇒ سيئ، تمرّر ⇒ جيد.

---

## F2 — اختبار الـCohesion (جملة واحدة)

**السؤال:** كيف تعرف في جملة واحدة أن التماسك **جيد**؟

**الجواب (جملة واحدة):** التماسك جيد إذا كانت عناصر الوحدة **تخدم مهمة وظيفية واحدة** بحيث تستطيع تسمية الوحدة بفعل واحد واضح (Functional) أو تربطها سلسلة تدفّق بيانات (Sequential)، والأسوأ كل ما كان تجميعها بدافع خارجي عن وظيفتها (زمن، صنف منطقي، أو صدفة).

**التشخيص السريع:** اسأل نفسك — *هل أستطيع تسمية هذه الوحدة بفعل واحد صريح؟* نعم ⇒ جيد، لا ⇒ ضعيف.

---

## F3 — الاختبار المزدوج (الجملتان معًا)

| # | البُعد | جملة التشخيص الواحدة | أفضل حالة | أسوأ حالة |
|---|---|---|---|---|
| 1 | Coupling | "هل تتشارك أم تمرّر؟" — تشارك ⇒ سيئ | Data (تمرير بيانات فقط) | Content (تغيير داخل وحدة أخرى) |
| 2 | Cohesion | "هل تُسمّى بفعل واحد؟" — نعم ⇒ جيد | Functional (مهمة وظيفية واحدة) | Coincidental (لا علاقة) |

**القاعدة النهائية (سلايد 31):** الهدف دائمًا **low coupling + high cohesion** — وهذا قرار واحد، مو اثنان: مكان حدّ الوحدة.

---

# خريطة التغطية (لا يوجد سؤال بلا مصدر)

| # | المفهوم | السؤال | المصدر |
|---|---|---|---|
| 1 | Design = HOW / problem-solving | Q1 | سلايد 2 |
| 2 | Conceptual vs technical design | Q2 | سلايد 5–7 |
| 3 | أهداف التصميم الأربعة | Q3 | سلايد 8 |
| 4 | Modularity / خصائص الوحدة | Q4 | سلايد 10–13 |
| 5 | Modularity–cost U-curve | Q5 | سلايد 14 (Fig. 4) |
| 6 | Abstraction | Q6 | سلايد 61 |
| 7 | Data vs procedural abstraction | Q7 | سلايد 41/46/61 |
| 8 | Information hiding / encapsulation | Q8 | سلايد 69 |
| 9 | Coupling تعريف + إمكانية الإلغاء | Q9 | سلايد 15–16 + تمرين 5.24 |

| # | المفهوم | السؤال | المصدر |
|---|---|---|---|
| 10 | أنواع Coupling الستة | Q10 | سلايد 19–21 (Fig. 7) |
| 11 | ليش Data أفضل و Content أسوأ | Q11 | سلايد 20/23 |
| 12 | Stamp vs Data coupling | Q12 | سلايد 20 + Fig. 6 سلايد 18 |
| 13 | Control coupling | Q13 | سلايد 21 + 17 |
| 14 | Common coupling | Q14 | سلايد 21 + Fig. 8 سلايد 22 |
| 15 | Content coupling | Q15 | سلايد 23 + Fig. 9 |
| 16 | Cohesion تعريف + الاتجاه | Q16 | سلايد 25 + Fig. 12 سلايد 31 |
| 17 | أنواع Cohesion السبعة | Q17 | سلايد 27 (Fig. 11) |
| 18 | Procedural vs Sequential | Q18 | سلايد 28/29 |
| 19 | Logical ⇒ Control coupling | Q19 | سلايد 30 + 21 + تمرين 5.8 |
| 20 | علاقة Coupling & Cohesion | Q20 | سلايد 31 (Fig. 12) |
| 21 | Top-down vs Bottom-up vs Hybrid | Q21 | سلايد 33–35 |
| 22 | Structure chart vs Flowchart | Q22 | سلايد 41 + تمرين 5.13 |
| 23 | SDD تعريف / blueprint | Q23 | سلايد 50–52 |
| 24 | SDD أربعة views | Q24 | سلايد 57 (Table 2) |
| 25 | عشرة entity attributes | Q25 | سلايد 51/53 |
| 26 | Object vs Class | Q26 | سلايد 60/62/63 |
| 27 | ADT vs Class | Q27 | سلايد 62/65 + تمرين 5.17 |
| 28 | Inheritance | Q28 | سلايد 67/68 |
| 29 | Inheritance vs Polymorphism | Q29 | سلايد 68/69 + تمرين 5.18 |
| 30 | Aggregation / Association / Dependency | Q30 | سلايد 77/78 |
| 31 | Use case model = Step 1 | Q31 | سلايد 72/73 |
| 32 | Use case vs Sequence diagram | Q32 | سلايد 73/75 |
| 33 | FOJ OOD orthogonal | Q33 | سلايد 58/36 |
| 34 | أسباب تحسّن OOD | Q34 | تمرين 5.19 + سلايد 58/68/69 |
| 35 | خطوات OOD الستة | Q35 | سلايد 72/75 (Fig. 25) |

---

## تحقق داخلي (Verification Notes)

**1) التطابق مع الحقائق المتحققة (Match with the frozen facts)**

| # | الحقيقة المتحققة (Fact) | مطابقة | المرجع |
|---|---|---|---|
| 1 | Coupling best → worst: Data > Stamp > Control > External > Common > Content | ✅ | Q10, Q11, F1 |
| 2 | Cohesion best → worst: Functional > Sequential > Communicational > Procedural > Temporal > Logical > Coincidental | ✅ | Q17, F2 |
| 3 | Strategies: Bottom-up / Top-down / Hybrid | ✅ | Q21 |
| 4 | أربعة أهداف للتصميم (four design objectives) | ✅ | Q3 |
| 5 | SDD = IEEE 1016؛ 4 views؛ 10 attributes | ✅ | Q23, Q24, Q25 |
| 6 | OOD = ست خطوات (six steps) | ✅ | Q31, Q35 |
| 7 | Modularity–cost = U-curve | ✅ | Q5 |

### 2) فحص تلخبط Coupling / Cohesion

| # | الفحص | النتيجة |
|---|---|---|
| 1 | كل سؤال coupling يذكر "between modules / interdependence" | لا يوجد تلخبط ✅ |
| 2 | كل سؤال cohesion يذكر "elements of a module / within module" | لا يوجد تلخبط ✅ |
| 3 | نطاق التغطية | Q9–Q15 coupling، Q16–Q20 cohesion |

### 3) ملاحظات عدم يقين (declared — Uncertainties)

| # | البند | ملاحظة |
|---|---|---|
| 1 | Communicational cohesion | مذكور في جدول Fig. 11 (سلايد 27) لكن نص الشرح التفصيلي (سلايد 26/28) يعدّد ستة أنواع فقط ولا يشرحه بكلماته — وُصِف صراحة في Q17 مع الإشارة لهذه الفجوة، ولم يُخترع له تعريف من خارج الفصل. |
| 2 | External coupling | مُدرَج في الترتيب (سلايد 19) لكن السلايدات لا تُفصّل تعريفه نصًّا كما تفعل مع البقية — وُصف بمستوى "ارتباط بصيغة/ترتيب خارجي" وهو الحد المفهوم من موقعه بين Control و Common. |
| 3 | SDD Table 1 | مذكور كعنوان فقط (سلايد 56) دون محتوى نصي في طبقة النص لـPDF — لم يُستشهد بمحتواه تفصيلًا. |

---

*المصدر: Singh & Aggarwal, Software Engineering, 3rd ed., Ch.5 — سلايدات 1–99 + كتاب ص.202–251. كل جواب مُتحقق منه من نص الفصل. بُني 2026-10-12.*